TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI (DNTU)
KHOA CÔNG NGHỆ THÔNG TIN – HỌC PHẦN WEB API (TH71108)

CẨM NANG LUYỆN NÓI & PHẢN BIỆN CÁ NHÂN
QUÁCH KHOA ĐỨC (1923040025)

Vai trò: Trưởng nhóm, Phụ trách Web Client & Kiến trúc tổng thể | Thứ tự thuyết trình: Người số 1 (Mở màn, Phần DỄ NHẤT) | Thời lượng mục tiêu: 2 phút 45 giây (Khóa cứng thời gian)
Dự án: Hệ thống Quản lý Hoạt động Hội nhập – VISIONARY

📌 **CHIẾN THUẬT DÀNH CHO ĐỨC:** Đức là người mở màn toàn bộ buổi bảo vệ. Nhiệm vụ cốt lõi: Nói to, rõ ràng, phong thái tự tin, trình bày đúng mục tiêu thực tế của CLB, chỉ rõ mô hình Một API - Ba Nền tảng và thực hiện một thao tác Web đơn giản. Tuyệt đối không đi sâu vào thuật toán mã hóa hay luồng xử lý phức tạp của Backend – phần đó đã có Hồ Khang và Minh Đạt đảm nhận.

I. PHÂN BỔ SLIDE VÀ THỜI GIAN (2 PHÚT 45 GIÂY)

Slide	Tiêu đề Slide	Nội dung trọng tâm	Thời lượng
Slide 1	Bìa & Giới thiệu	Chào hội đồng, giới thiệu tên đề tài Visionary, 4 thành viên và địa chỉ API Swagger	35 giây
Slide 2	Bài toán thực tế & Mục tiêu	Vấn đề quản lý rời rạc của CLB Hội nhập và 3 mục tiêu kết nối sinh viên	40 giây
Slide 3	Kiến trúc Một API, Ba Ứng dụng	Sơ đồ kiến trúc: Web, WinForms, Android cùng trỏ vào 1 ASP.NET Core API duy nhất	45 giây
Slide 4	Ứng dụng Web Client (Next.js)	Giao diện Web, các điều khiển chính, mã nguồn gọi GET /api/events & demo live	45 giây

II. LỜI THUYẾT TRÌNH TỪNG CÂU (HỌC THUỘC LÒNG & NÓI TỰ NHIÊN)

1. Khi ở Slide 1 (Bìa) – 35 giây:

"Kính chào Thầy và các bạn trong hội đồng. Em tên là Quách Khoa Đức, đại diện nhóm phát triển đề tài: 'Hệ thống Quản lý Hoạt động Hội nhập – VISIONARY' trong học phần Web API TH71108.

Nhóm chúng em gồm bốn thành viên: Em Quách Khoa Đức phụ trách mở đầu, kiến trúc và Web Client; bạn Đào Trí Khang phụ trách ứng dụng Android; bạn Hồ Đoàng Nguyên Khang phụ trách Web API,

Swagger và Cơ sở dữ liệu; và bạn Võ Ngọc Minh Đạt phụ trách ứng dụng Windows Forms cùng phân tích sự cố.

Hệ thống của nhóm đã được triển khai hoàn chỉnh. Backend Web API đang chạy trực tiếp trên cloud tại địa chỉ có thể mở ngay Swagger để kiểm chứng. Trong quá trình thực hiện, nhóm có sử dụng công cụ AI Google Antigravity để hỗ trợ rà soát mã nguồn và kiểm thử tự động, toàn bộ logic nghiệp vụ đều do nhóm trực tiếp kiểm chứng và làm chủ."

2. Khi ở Slide 2 (Bài toán & Mục tiêu) – 40 giây:

"Thưa Thầy, xuất phát từ thực tế Câu lạc bộ Hội nhập tại trường, việc truyền thông sự kiện, đăng ký tham gia và điểm danh trước đây thường dùng Google Forms hoặc mạng xã hội. Dữ liệu bị phân tán, sinh viên khó theo dõi cơ hội việc làm, còn Ban Chủ nhiệm thì mất nhiều giờ tổng hợp thủ công.

Vì vậy, mục tiêu của VISIONARY là xây dựng một nền tảng quản lý tập trung với ba mục tiêu chính:
1. Quản lý hoạt động, sự kiện và tài liệu hội nhập đồng bộ.
2. Kết nối sinh viên nhanh chóng trên các thiết bị quen thuộc: từ máy tính cho tới điện thoại di động.
3. Sử dụng DUY NHẤT một Web API trung tâm để đảm bảo dữ liệu luôn nhất quán, không xảy ra xung đột."

3. Khi ở Slide 3 (Kiến trúc Một API, Ba Ứng dụng) – 45 giây:

"Trên màn hình là sơ đồ kiến trúc tổng quan của toàn bộ hệ thống. Ở trung tâm là Web API viết bằng ASP.NET Core 8, kết nối với cơ sở dữ liệu PostgreSQL lưu trữ trên Supabase qua Entity Framework Core.

Ba ứng dụng độc lập kết nối vào Web API này gồm:
- Ứng dụng Web viết bằng Next.js và TypeScript, chạy trên trình duyệt người dùng.
- Ứng dụng Windows Forms viết bằng C#, chạy trên máy tính dành cho Ban Chủ nhiệm quản trị.
- Ứng dụng Mobile Android viết bằng Java Native, chạy trên điện thoại sinh viên.

Cả ba ứng dụng đều giao tiếp với API qua giao thức HTTPS bằng định dạng JSON chuẩn. Nhờ kiến trúc này, khi Ban Chủ nhiệm tạo một sự kiện trên Web hoặc điểm danh trên WinForms, sinh viên trên Android sẽ thấy kết quả ngay lập tức mà không cần mỗi nền tảng phải tự dựng một cơ sở dữ liệu riêng."

4. Khi ở Slide 4 (Web Client & Thao tác Demo) – 45 giây:

"Về ứng dụng Web, nhóm xây dựng bằng Next.js với thiết kế giao diện sản phẩm thực tế, hướng đến người dùng cuối. Các điều khiển chính gồm thanh điều hướng, ô tìm kiếm sự kiện và lưới hiển thị thẻ sự kiện.

[ĐỨC THỰC HIỆN THAO TÁC LIVE TRÊN TRÌNH DUYỆT: Mở trang Web, bấm vào mục 'Sự kiện', chỉ vào danh sách sự kiện đang hiện ra].

Như Thầy và các bạn thấy, khi em bấm vào tab Sự kiện, trình duyệt gửi một yêu cầu GET /api/events đến Web API. API truy vấn cơ sở dữ liệu và trả về JSON danh sách sự kiện kèm phân trang. JavaScript trên Web nhận dữ liệu và ánh xạ thành các thẻ sự kiện trực quan.

Khó khăn lớn nhất của Web là xử lý bất đồng bộ và đồng bộ trạng thái khi có phân trang, nhóm đã giải quyết bằng useEffect và khối quản lý trạng thái tập trung.

Tiếp theo, em xin trân trọng chuyển lời cho bạn Đào Trí Khang để trình bày về ứng dụng Android và cách thức kết nối API trên thiết bị di động. Xin mời Khang!"

III. MÃ NGUỒN CẦN MỞ VÀ 3 CHỖ GIẢNG VIÊN DỄ TRỎ VÀO

Tệp mã nguồn Đức phụ trách mở khi giảng viên yêu cầu kiểm tra code Web:

📁 Đường dẫn: frontend/src/services/eventService.ts

📁 Hoặc tệp giao diện: frontend/src/app/events/page.tsx

📘 **Mã nguồn minh chứng: Hàm getEvents trong frontend/src/services/eventService.ts**

```
// frontend/src/services/eventService.ts
export const getEvents = async (page: number = 1, pageSize: number = 10, category?: string) => {
  const params = new URLSearchParams({ page: page.toString(), pageSize: pageSize.toString() });
  if (category) params.append('category', category);

  // Gửi request GET tới API thông qua hàm fetch tập trung
  const response = await fetchWithAuth(`/api/events?${params.toString()}`, {
    method: 'GET'
  });
  return response; // Trả về ApiResponse<PagedResponse<EventItem>>
};
```

Ba điểm nóng trên dòng code giảng viên có thể hỏi:

👉 Điểm 1: Dòng 'await fetchWithAuth(...)':

Giảng viên hỏi: Dòng này làm gì? fetchWithAuth khác gì fetch thông thường?
-> Đức trả lời: 'Dạ thưa Thầy, dòng này gửi yêu cầu HTTP GET lên server. Hàm fetchWithAuth là hàm dùng chung do nhóm tự viết, nó có nhiệm vụ tự động kiểm tra và gắn Bearer Token vào header Authorization nếu người dùng đã đăng nhập, giúp các trang khác không phải viết lại logic này.'

👉 Điểm 2: Dòng 'new URLSearchParams(...)':

Giảng viên hỏi: Tại sao phải dùng URLSearchParams mà không ghép chuỗi thủ công?
-> Đức trả lời: 'Dạ thưa Thầy, URLSearchParams tự động mã hóa an toàn các tham số truy vấn như dấu cách và ký tự tiếng Việt thành chuẩn URL an toàn (URL encoding), tránh lỗi cú pháp và nguy cơ tấn công Injection vào đường dẫn URL.'

👉 Điểm 3: Chỗ giao diện nhận dữ liệu (setEvents):

Giảng viên hỏi: Dữ liệu từ API trả về dạng nào và giao diện đổ lên ra sao?
-> Đức trả lời: 'Dạ API trả về đối tượng ApiResponse bọc PagedResponse có trường items chứa mảng danh sách sự kiện. Frontend dùng React useState để lưu mảng items này, sau đó dùng hàm .map() để dựng các thẻ Card sự kiện trên màn hình ạ.'

IV. BỘ CÂU HỎI PHẢN BIỆN CHÍNH & PHỤ LỤC HỎI CHÉO (TRẢ LỜI TRONG 20-30 GIÂY)

❓ Câu 1: Web API là gì và tại sao nhóm lại cần dùng chung một Web API?

🗣️ Đức trả lời (20-30s): Dạ thưa Thầy, Web API là giao diện lập trình ứng dụng qua giao thức HTTP, đóng vai trò là một 'hợp đồng' trung gian chuẩn hóa trao đổi dữ liệu. Nhóm dùng chung một API để cả Web, WinForms và Android cùng truy xuất một nguồn dữ liệu duy nhất, đảm bảo tính nhất quán và bảo mật tập trung.

❓ Câu 2: Frontend Web của nhóm lưu Token đăng nhập ở đâu? Có an toàn không?

🗣️ Đức trả lời (20-30s): Dạ thưa Thầy, trên Web nhóm lưu JWT Token trong localStorage (hoặc Secure Cookie) để duy trì phiên làm việc khi người dùng F5 hoặc mở tab mới. Để an toàn, Token chỉ chứa định danh và vai trò, tuyệt đối không chứa mật khẩu; hạn dùng của token là 4 giờ và mọi API nhạy cảm đều được Server xác thực lại chữ ký.

❓ Câu 3: CORS là gì? Tại sao chỉ ứng dụng Web cần cấu hình CORS mà WinForms với Android thì không?

🗣️ Đức trả lời (20-30s): Dạ thưa Thầy, CORS là cơ chế bảo mật của TRÌNH DUYỆT (Browser Same-Origin Policy) nhằm ngăn chặn mã JavaScript từ một tên miền này tự ý gọi tới tên miền khác. WinForms và Android là ứng dụng Native chạy trực tiếp trên hệ điều hành, không bị chi phối bởi cơ chế của trình duyệt nên không cần quan tâm đến CORS.

❓ Câu 4: Khi người dùng bấm F5 hoặc tắt tab thì điều gì xảy ra với trạng thái Web?

🗣️ Đức trả lời (20-30s): Dạ thưa Thầy, khi tắt tab hoặc F5, bộ nhớ RAM của trang web bị xóa, nhưng Token đã được lưu trong localStorage nên khi mở lại, ứng dụng tự đọc lại Token và gửi lên API để khôi phục phiên đăng nhập mà người dùng không cần gõ lại mật khẩu.

❓ Câu 5 (Hỏi chéo): Nếu Web API bị sập thì cả ba ứng dụng sẽ thế nào?

🗣️ Đức trả lời (20-30s): Dạ thưa Thầy, nếu API sập, cả ba ứng dụng đều không thể đọc ghi dữ liệu từ cơ sở dữ liệu. Nhưng ứng dụng không bị văng crash mà sẽ bắt lỗi mạng (mã trạng thái 0 hoặc 500) và hiển thị thông báo thân thiện cho người dùng là 'Không thể kết nối đến máy chủ, vui lòng thử lại sau'.

V. CHECKLIST TỰ LUYỆN TRƯỚC BUỔI BẢO VỆ

- [ ] Đã tập nói trôi chảy Slide 1 đến 4, bấm giờ đúng dưới 2 phút 45 giây.
- [ ] Mở sẵn trình duyệt vào trang Web Visionary: https://visionary-platform.vercel.app/events.
- [ ] Mở sẵn VS Code trỏ đúng vào frontend/src/services/eventService.ts.
- [ ] Nhớ câu chuyển lời chuẩn: 'Tiếp theo em xin trân trọng chuyển lời cho bạn Đào Trí Khang...'
- [ ] Đã nắm chắc câu trả lời ngắn 20-30 giây cho câu hỏi về CORS và lưu Token trên Web.